OAVS COMPUTER TEACHER (TGT) • DBMS STUDY NOTES

Chapter 6

Relational Database

Topics Covered

1. 6.1 Introduction to Relational Database
2. Codd's 12 Rules
3. Concept of Domain, Tuple, Cardinality
4. Comparison: HDB vs NDB vs RDB

Concise • Exam-Focused • Diagram-Supported Notes

6.1 Introduction to Relational Database

A **Relational Database (RDB)** stores data in the form of **relations** (tables), where each table consists of rows and columns. The model was proposed by **Dr. Edgar F. Codd** in **1970** at IBM, in his paper "A Relational Model of Data for Large Shared Data Banks." It is based on strong mathematical foundations of **set theory** and **predicate logic**.

> **Exam Point:** E.F. Codd → Father of the Relational Database Model. Year → 1970. Company → IBM.

Basic Terminology

Term	Formal Name	Meaning
Table	Relation	A collection of related data organised in rows & columns
Row	Tuple	A single record of the relation
Column	Attribute	A named property/field of the relation
No. of Columns	Degree	Total number of attributes in a relation
No. of Rows	Cardinality	Total number of tuples in a relation
Value Set	Domain	Set of permissible values for an attribute — e.g., Gender ∈ {Male, Female, Other}

Relation: STUDENT

Roll_No	Name	Age	City
101	Anita	21	Bhubaneswar
102	Rahul	22	Sambalpur
103	Sneha	21	Puri

↑ Attributes (Columns) — Degree = 4

↑ Tuples (Rows) — Cardinality = 3

Fig 6.1 — Structure of a Relation showing Attributes, Tuples, Degree and Cardinality

Characteristics of a Relational Database

- Data is stored in **tables** (relations); each table has a unique name.
- Each **row (tuple)** is unique and represents one record.
- Each **column (attribute)** holds atomic (indivisible) values from a defined domain.
- Order of rows and columns is **not significant**.
- Relationships between tables are established using **common columns (keys)**, not physical pointers.
- Supported by a standard query language — **SQL**.

Advantages

- Simple, table-based structure — easy to understand and use.
- Data independence (logical & physical).
- Reduces data redundancy through normalisation.
- Ensures data integrity using keys and constraints.
- Flexible querying using SQL.

Limitations

- May become slower when handling extremely large volumes of unstructured data (like images, videos, or free-form text) compared to specialised databases.
- Designing a good relational schema requires proper planning and knowledge of normalisation rules; a poorly designed schema can still have redundancy issues.

Remember: RDB is based on the **Relational Model**, which represents data and relationships purely as tables — there are no navigational pointers as in hierarchical or network models.

Codd's 12 Rules (for a truly Relational DBMS)

To test whether a DBMS is **truly relational**, Dr. E.F. Codd proposed a set of rules in 1985. Although called "12 Rules," there are actually **13 rules (Rule 0 to Rule 12)** — Rule 0 is the foundation rule on which all others depend.

Exam Point: Total Rules = 13 (Rule 0 to Rule 12). Commonly asked as "Codd's 12 Rules." No commercial RDBMS satisfies all 13 rules completely.

0 Foundation Rule

The system must manage its stored and displayed data **using only its relational capabilities** — i.e., the DBMS must be relational at the core, not just on the surface.

1 Information Rule

All information in the database (including table names, column names, values) must be represented explicitly as **values in tables** — no other way (like separate files) is allowed.

2 Guaranteed Access Rule

Every single data value must be accessible by specifying: **Table Name + Primary Key value + Column Name**. Example: `STUDENT[Roll_No=101].Name`.

3 Systematic Treatment of NULL Values

NULL values (missing/unknown data) must be supported and treated **systematically**, independent of data type, and distinct from zero or blank string.

4 Active/Dynamic Online Catalog

The database's structure (schema/metadata) must itself be stored in tables and be **queryable using the same query language** used for normal data.

5 Comprehensive Data Sub-language Rule

The system must support at least **one language** (e.g., SQL) that can handle data definition, manipulation, integrity constraints, authorization, and transactions — all with a well-defined syntax.

6 View Updating Rule

All **views** that are theoretically updatable must also be updatable by the system.

7 High-Level Insert, Update, Delete

The system must support **set-level** INSERT, UPDATE and DELETE operations — i.e., operate on a whole set of rows at once, not just one row at a time.

8 Physical Data Independence

Changes in the **physical storage** (file structure, storage device, indexing method) should **not affect** application programs or user access.

9 Logical Data Independence

Changes in the **logical structure** (adding/removing tables or columns) should not force changes to existing application programs, as far as possible.

10 Integrity Independence

Integrity constraints (e.g., Primary Key, Not Null) must be definable in the **catalog/schema**, not hard-coded inside application programs.

11 Distribution Independence

Users should not notice — and applications should not be affected — whether the database is **centralised or distributed** across multiple locations.

12 Non-subversion Rule

If the system provides a low-level (record-at-a-time) interface, it must **not be able to bypass** the integrity rules and constraints enforced by the higher-level relational language.

> **Shortcut to Remember (mnemonic order):** Foundation → Information → Guaranteed Access → Nulls → Catalog → Sub-language → View Update → Set-level Ops → Physical Independence → Logical Independence → Integrity → Distribution → Non-subversion.

Quick Revision Table — Codd's Rules

Rule	One-line Idea
0	DBMS must be relational at its core
1	All data represented as table values
2	Access data via table + key + column
3	Proper support for NULL values
4	Metadata stored & queried like data
5	One language for all operations (SQL)

6	Updatable views must be updatable
7	Set-based insert/update/delete
8	Independent of physical storage changes
9	Independent of logical schema changes
10	Constraints stored in catalog, not code
11	Works same whether centralised/distributed
12	Low-level access cannot bypass integrity rules

Common Mistake: Students often forget Rule 0 and answer "12 rules" as Rule 1–12 only. Always mention that the **Foundation Rule (Rule 0)** makes the total 13.

Concept of Domain, Tuple and Cardinality

Domain

A **Domain** is the set of all **permissible (legal) values** that an attribute can take. Every attribute in a relation is defined over a domain.

Attribute	Domain (Set of Allowed Values)
Gender	{Male, Female, Other}
Age	Integer values between 0 and 120
Roll_No	Positive integers
City	Set of valid city names (text)

Exam Point: Domain defines both the **data type** and the **valid range/set of values** — it prevents entry of meaningless data (e.g., Age = "Blue" is invalid because it is outside the Age domain).

Tuple

A **Tuple** is a single **row** of a relation, representing one complete record — an ordered set of attribute values, one drawn from each attribute's domain.

Remember: Number of tuples = number of rows = **Cardinality** of the relation. This value keeps **changing** as records are inserted or deleted.

Cardinality

Cardinality is the **total number of tuples (rows)** present in a relation at a given time.

Relation: BOOK

Book_ID	Title	Price
B001	DBMS Basics	450
B002	Data Structures	380
B003	Networks	520

Degree = 3
(3 attributes)
Cardinality = 3
(3 tuples/rows)

Each tuple = one row (e.g., B001, "DBMS Basics", 450) drawn from the domains of Book_ID, Title, Price.

Fig 6.2 — Domain, Tuple, Degree and Cardinality illustrated together

Summary Table

Concept	Definition	Example (BOOK table)
Domain	Set of valid/allowed values for an attribute	Price ∈ 0–9999
Tuple	A single row / record of the relation	(B001, "DBMS Basics", 450)
Cardinality	Number of tuples (rows) in the relation	3
Degree	Number of attributes (columns) in the relation	3

Common Mistake: Do not confuse **Cardinality** (number of rows) with **Degree** (number of columns). Also, cardinality of a relation ≠ cardinality of a relationship (mapping like 1:1, 1:M) — this chapter deals only with **relation cardinality** (row count).

Shortcut: "Tuple = Table's row, Domain = Data's Range, Cardinality = Count of rows, Degree = Count of columns."

Comparison: Hierarchical (HDB) vs Network (NDB) vs Relational (RDB) Database

Structural Overview

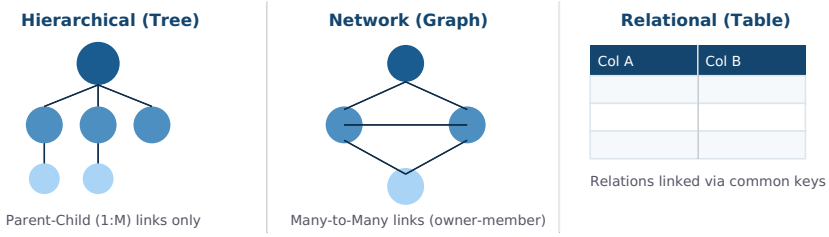


Fig 6.3 — HDB (tree), NDB (graph), and RDB (independent tables linked by keys)

Detailed Comparison Table

Feature	Hierarchical DB (HDB)	Network DB (NDB)	Relational DB (RDB)
Structure	Tree-like (parent–child)	Graph-like (owner–member)	Table-based (rows & columns)
Relationship Type	Only One-to-Many (1:M)	Many-to-Many (M:N) supported	All types via keys (1:1, 1:M, M:N)
Navigation	Top-down, via physical pointers	Via pointers/links between records	Via SQL queries, no navigation needed
Data Redundancy	High (child repeated under each parent)	Medium (still uses links)	Low (controlled via normalisation)
Flexibility	Rigid; hard to restructure	Complex but more flexible than HDB	Highly flexible & easy to modify
Ease of Use	Simple for strict hierarchies only	Complex, needs knowledge of pointers	Easy — uses simple table & SQL concepts

Data Independence	Low	Low to Medium	High (logical + physical)
Integrity Constraints	Limited support	Limited support	Strong support (keys, constraints)
Query Language	Procedural (record-at-a-time)	Procedural (record-at-a-time)	Declarative (SQL — set-at-a-time)
Example Systems	IBM IMS	IDMS	MySQL, Oracle, PostgreSQL, SQL Server
Best Suited For	Simple, strictly hierarchical data (e.g., org chart)	Complex relationships with shared ownership	General-purpose, most modern applications

Exam Point: Evolution order → Hierarchical (1960s) → Network (late 1960s) → **Relational (1970, by Codd)**. RDB replaced HDB/NDB mainly because it removed dependency on physical pointers and offered data independence.

Common Mistake: Do not say "Network Database supports only Many-to-Many." NDB supports 1:1, 1:M and M:N — its key strength over HDB is M:N support.

⚡ Quick Revision / Cheat Sheet — Chapter 6

- **RDB proposed by:** E.F. Codd, 1970, IBM.
- **Relation** = Table | **Tuple** = Row | **Attribute** = Column.
- **Degree** = No. of columns | **Cardinality** = No. of rows.
- **Domain** = Set of permissible values for an attribute.
- **Codd's Rules:** Total = 13 (Rule 0 to Rule 12); commonly called "12 Rules."
- Rule 0 = Foundation; Rule 1 = Information; Rule 2 = Guaranteed Access; Rule 3 = Nulls; Rule 4 = Active Catalog; Rule 5 = Data Sub-language; Rule 6 = View Updating; Rule 7 = High-level Insert/Update/Delete; Rule 8 = Physical Independence; Rule 9 = Logical Independence; Rule 10 = Integrity Independence; Rule 11 = Distribution Independence; Rule 12 = Non-subversion.
- **HDB** → Tree structure, only 1:M relationships (e.g., IBM IMS).
- **NDB** → Graph structure, supports M:N relationships (e.g., IDMS).
- **RDB** → Table structure, links via keys, supports all relationship types, uses SQL.

📌 Important One-Line Facts

- A Domain restricts an attribute to only meaningful, valid values.
- Cardinality of a relation changes with every INSERT or DELETE operation.
- Degree of a relation normally remains fixed unless the schema (structure) is altered.
- No commercial RDBMS fully satisfies all 13 of Codd's Rules.
- RDB uses a declarative, set-based query language (SQL); HDB/NDB use procedural, record-at-a-time access.
- The Relational Model has no physical pointers — relationships are shown only through matching key values.